

Geometry

Point Location Test

```
#include

using namespace std;
typedef long long ll;

struct Point {
    int x, y;
    void read(){ scanf("%d %d", &x, &y); }
    Point operator +(const Point& b) const { return Point{x+b.x, y+b.y}; }
    Point operator -(const Point& b) const { return Point{x-b.x, y-b.y}; }
    ll operator *(const Point& b) const { return (ll) x * b.y - (ll) y * b.x; }
    void operator +=(const Point& b) { x += b.x; y += b.y; }
    void operator -=(const Point& b) { x -= b.x; y -= b.y; }
    void operator *=(const int k) { x *= k; y *= k; }

    ll cross(const Point& b, const Point& c) const {
        return (b - *this) * (c - *this);
    }
};

int T;
Point P[3];

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        for(int i = 0; i < 3; i++)
            P[i].read();

        ll cross = P[0].cross(P[1], P[2]);
        if(cross < 0) printf("RIGHT\n");
        else if(cross > 0) printf("LEFT\n");
        else printf("TOUCH\n");
    }
}
```

Line Segment Intersection

```
#include

using namespace std;
typedef long long ll;

struct Point {
    int x, y;
    void read(){ scanf("%d %d", &x, &y); }
    Point operator +(const Point& b) const { return Point{x+b.x, y+b.y}; }
    Point operator -(const Point& b) const { return Point{x-b.x, y-b.y}; }
    ll operator *(const Point& b) const { return (ll) x * b.y - (ll) y * b.x; }
    void operator +=(const Point& b) { x += b.x; y += b.y; }
```

```

void operator -=(const Point &b) { x -= b.x; y -= b.y; }
void operator *=(const int k) { x *= k; y *= k; }

ll cross(const Point& b, const Point& c) const {
    return (b - *this) * (c - *this);
}
};

int T;
Point P[4];

void lineintersect(Point P1, Point P2, Point P3, Point P4){
    // Parallel case
    if((P2-P1) * (P4-P3) == 0){
        // Collinear case, check bounding boxes
        if(P1.cross(P2, P3) == 0){
            for(int i = 0; i < 2; i++){
                if(max(P1.x, P2.x) < min(P3.x, P4.x) || max(P1.y, P2.y) < min(P3.y, P4.y)){
                    printf("NO\n");
                    return;
                }
                swap(P1, P3);
                swap(P2, P4);
            }
            printf("YES\n");
            return;
        }

        // Non-collinear parallel lines never intersect
        printf("NO\n");
        return;
    }

    // Non-parallel case
    for(int i = 0; i < 2; i++){
        ll s1 = P1.cross(P2, P3);
        ll s2 = P1.cross(P2, P4);
        if((s1 < 0 && s2 < 0) || (s1 > 0 && s2 > 0)){
            printf("NO\n");
            return;
        }
        swap(P1, P3);
        swap(P2, P4);
    }

    printf("YES\n");
}

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        for(int i = 0; i < 4; i++)
            P[i].read();
        lineintersect(P[0], P[1], P[2], P[3]);
    }
}

```

Polygon Area

```
#include
```

```
using namespace std;
typedef long long ll;
const int maxN = 1001;

int N;
ll x[maxN], y[maxN], ans;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%lld %lld", &x[i], &y[i]);
        x[N] = x[0]; y[N] = y[0];

        for(int i = 0; i < N; i++){
            ans += x[i] * y[i+1];
            ans -= y[i] * x[i+1];
        }
        printf("%lld\n", abs(ans));
    }
}
```

Point in Polygon

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1001;

struct Point {
    int x, y;
    void read(){ scanf("%d %d", &x, &y); }
    Point operator +(const Point& b) const { return Point{x+b.x, y+b.y}; }
    Point operator -(const Point& b) const { return Point{x-b.x, y-b.y}; }
    ll operator *(const Point& b) const { return (ll) x * b.y - (ll) y * b.x; }
    void operator +=(const Point& b) { x += b.x; y += b.y; }
    void operator -=(const Point& b) { x -= b.x; y -= b.y; }
    void operator *=(const int k) { x *= k; y *= k; }

    ll cross(const Point& b, const Point& c) const {
        return (b - *this) * (c - *this);
    }
};

int N, M;
Point P[maxN];

bool pointlineintersect(Point P1, Point P2, Point P3){
    if(P2.cross(P1, P3) != 0) return false;
    return (min(P2.x, P3.x) <= P1.x && P1.x <= max(P2.x, P3.x))
        && (min(P2.y, P3.y) <= P1.y && P1.y <= max(P2.y, P3.y));
}

void pointinpolygon(){
    int cnt = 0;
    bool boundary = false;
    for(int i = 1; i <= N; i++){
        int j = (i == N ? 1 : i+1);
        if(pointlineintersect(P[0], P[i], P[j]))
            boundary = true;

        if(P[i].x <= P[0].x && P[0].x < P[j].x && P[0].cross(P[i], P[j]) < 0) cnt++;
    }
}
```

```

        else if(P[j].x <= P[0].x && P[0].x < P[i].x && P[0].cross(P[j], P[i]) < 0) cnt++;
    }

    if(boundary) printf("BOUNDARY\n");
    else if(cnt&1) printf("INSIDE\n");
    else printf("OUTSIDE\n");
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 1; i <= N; i++)
        P[i].read();
    for(int i = 0; i < M; i++){
        P[0].read();
        pointinpolygon();
    }
}

```

Polygon Lattice Points

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+5;

int N;
ll area, boundary, inside, x[maxN], y[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%lld %lld", &x[i], &y[i]);
    x[N] = x[0]; y[N] = y[0];

    for(int i = 0; i < N; i++){
        area += x[i] * y[i+1];
        area -= y[i] * x[i+1];
    }
    area = abs(area);

    for(int i = 0; i < N; i++){
        if(x[i+1] == x[i]) boundary += abs(y[i+1]-y[i]);
        else if(y[i+1] == y[i]) boundary += abs(x[i+1]-x[i]);
        else boundary += __gcd(abs(x[i+1]-x[i]), abs(y[i+1]-y[i]));
    }

    printf("%lld %lld\n", (area+2-boundary)/2, boundary);
}

```

Minimum Euclidean Distance

```

#include

using namespace std;
typedef long long ll;
typedef pair pll;
const int maxN = 2e5+1;

int N;
ll best;
bool isLeft[maxN];
pll P[maxN];

ll square(ll x){
    return x*x;
}

ll dist(pll A, pll B){
    ll dx = A.first-B.first;
    ll dy = A.second-B.second;
    return square(dx) + square(dy);
}

void solve(vector PX, vector PY){
    int len = PX.size();
    if(len == 1) return;

    vector LX, LY, RX, RY;
    for(int i = 0; i < len; i++){
        if(i < len/2){
            LX.push_back(PX[i]);
            isLeft[PX[i]] = true;
        } else {
            RX.push_back(PX[i]);
            isLeft[PX[i]] = false;
        }
    }

    for(int id : PY){
        if(isLeft[id]) LY.push_back(id);
        else RY.push_back(id);
    }

    solve(LX, LY);
    solve(RX, RY);

    ll midX = P[LX.back()].first;
    vector stripe;
    for(int id : PY)
        if(square(P[id].first-midX) < best)
            stripe.push_back(id);

    for(int i = 0; i < (int) stripe.size(); i++)
        for(int j = i+1; j < (int) stripe.size() && square(P[stripe[i]].second-P[stripe[j]].second) < best; j++)
            best = min(best, dist(P[stripe[i]], P[stripe[j]]));
}

int main(){
    scanf("%d", &N);
    vector sortedX, sortedY;
    for(int i = 0, x, y; i < N; i++){
        scanf("%d %d", &x, &y);
        P[i] = {x, y};
        sortedX.push_back(i);
        sortedY.push_back(i);
    }
    sort(sortedX.begin(), sortedX.end(), [](int A, int B){
        return P[A].first < P[B].first;
    });
    sort(sortedY.begin(), sortedY.end(), [](int A, int B){

```

```

        return P[A].second < P[B].second;
    });

    best = LLONG_MAX;
    solve(sortedX, sortedY);
    printf("%lld\n", best);
}

```

Convex Hull

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+5;

struct Point {
    int x, y;
    void read(){ scanf("%d %d", &x, &y); }
    Point operator +(const Point& b) const { return Point{x+b.x, y+b.y}; }
    Point operator -(const Point& b) const { return Point{x-b.x, y-b.y}; }
    ll operator *(const Point& b) const { return (ll) x * b.y - (ll) y * b.x; }
    bool operator <(const Point& b) const { return x == b.x ? y < b.y : x < b.x; }
    void operator +=(const Point& b) { x += b.x; y += b.y; }
    void operator -=(const Point& b) { x -= b.x; y -= b.y; }
    void operator *=(const int k) { x *= k; y *= k; }

    ll cross(const Point& b, const Point& c) const {
        return (b - *this) * (c - *this);
    }
};

int N, S;
Point P[maxN];
vector hull;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        P[i].read();
    }
    sort(P, P+N);

    for(int t = 0; t < 2; t++){
        for(int i = 0; i < N; i++){
            while((int) hull.size()-S >= 2){
                Point P1 = hull[hull.size()-2];
                Point P2 = hull[hull.size()-1];
                if(P1.cross(P2, P[i]) <= 0) break;
                hull.pop_back();
            }
            hull.push_back(P[i]);
        }
        hull.pop_back();
        S = hull.size();
        reverse(P, P+N);
    }

    printf("%d\n", S);
    for(Point h : hull)

```

```

        printf("%d %d\n", h.x, h.y);
    }

```

Maximum Manhattan Distances

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f3f;

int N;
ll l1, l2, l3, l4;

int main(){
    scanf("%d", &N);

    l1 = l4 = -INF;
    l2 = l3 = INF;

    for(int i = 0; i < N; i++){
        ll x, y;
        scanf("%lld %lld", &x, &y);

        l1 = max(l1, x+y);
        l2 = min(l2, x+y);
        l3 = min(l3, x-y);
        l4 = max(l4, x-y);

        printf("%lld\n", max(l1-l2, l4-l3));
    }
}

```

All Manhattan Distances

```

#include

using namespace std;
const int maxN = 2e5+1;

int N, x[maxN], y[maxN];
__int128 xcur, ycur, ans;

ostream& operator<<(ostream& o, const __int128& x) {
    if(x == numeric_limits<__int128>::min())
        return o << "-170141183460469231731687303715884105728";
    else if(x < 0) return o << "-" << -x;
    else if(x < 10) return o << (char)(x + '0');
    else return o << x / 10 << (char)(x % 10 + '0');
}

int main(){

```

```

scanf("%d", &N);
for(int i = 0; i < N; i++){
    scanf("%d %d", &x[i], &y[i]);

    sort(x, x+N);
    sort(y, y+N);

    xcur = ycur = ans = 0;
    for(int i = 1; i < N; i++){
        xcur += (__int128) (x[i] - x[i-1]) * i;
        ycur += (__int128) (y[i] - y[i-1]) * i;
        ans += xcur + ycur;
    }
    cout << ans << endl;
}

```

Intersection Points

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5+1;
const int maxX = 1e6+5;
const int SIZE = 2*maxX;

struct Operation {
    int type, y, a, b;
};

int N, ds[SIZE];
vector ops;

void update(int idx, int val){
    for(int i = idx; i < SIZE; i += -i&i)
        ds[i] += val;
}

int query(int idx){
    int sum = 0;
    for(int i = idx; i > 0; i -= -i&i)
        sum += ds[i];
    return sum;
}

int main(){
    scanf("%d", &N);
    for(int i = 0, a, b, c, d; i < N; i++){
        scanf("%d %d %d %d", &a, &b, &c, &d);
        if(a == c){
            // Vertical
            ops.push_back({2, b, a+maxX, -1});
            ops.push_back({3, d, a+maxX, -1});
        } else {
            // Horizontal
            ops.push_back({1, b, a+maxX, c+maxX});
        }
    }
    sort(ops.begin(), ops.end(), [](Operation A, Operation B){
        return A.y < B.y;
    });
}

```



```

ll ans = 0;
for(Operation O : ops){
    if(O.type == 1)        ans += query(O.b) - query(O.a-1);
    else if(O.type == 2)    update(O.a, 1);
    else if(O.type == 3)    update(O.a, -1);
}
printf("%lld\n", ans);
}

```

Line Segments Trace I

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Line {
    ll a, b;

    Line() : a(0), b(-INF) {}
    Line(ll a, ll b) : a(a), b(b) {}

    ll get(ll x){
        return a * x + b;
    }
};

struct Node {
    Line line = Line();
    Node* lc = nullptr;
    Node* rc = nullptr;
};

struct LiChaoTree {
    Node* root;
    ll size;
    LiChaoTree(ll size) : root(nullptr), size(size) {}

private:
    void contained_insert(Node* &node, ll tl, ll tr, Line line){
        if(node == nullptr) node = new Node();
        if(node->line.get(tl) < line.get(tl)) swap(node->line, line);
        if(node->line.get(tr) >= line.get(tr)) return;
        if(tl == tr) return;

        ll tm = (tl + tr) / 2;
        if(node->line.get(tm) > line.get(tm)) {
            contained_insert(node->rc, tm+1, tr, line);
        } else {
            swap(node->line, line);
            contained_insert(node->lc, tl, tm, line);
        }
    }

    void insert(Node* &node, ll tl, ll tr, ll l, ll r, Line line){
        if(tr < l || r < tl || tl > tr || l > r) return;
        if(node == nullptr) node = new Node();
        if(l <= tl && tr <= r) return contained_insert(node, tl, tr, line);
    }
};

```

```

    ll tm = (tl + tr) / 2;
    insert(node->lc, tl, tm, l, r, line);
    insert(node->rc, tm+1, tr, l, r, line);
}

ll query(Node* &node, ll tl, ll tr, ll x) {
    if(node == nullptr) return -INF;
    if(tl == tr) return node->line.get(x);

    ll res = node->line.get(x);
    ll tm = (tl + tr) / 2;
    if(x <= tm) res = max(res, query(node->lc, tl, tm, x));
    else res = max(res, query(node->rc, tm+1, tr, x));

    return res;
}

public:
    void insert(ll l, ll r, Line line) {
        insert(root, 0, size-1, l, r, line);
    }

    ll query(ll x) {
        return query(root, 0, size-1, x);
    }
};

int main(){
    int N, M;
    scanf("%d %d", &N, &M);
    LiChaoTree tree(M+1);

    for(int i = 0, y1, y2; i < N; i++){
        scanf("%d %d", &y1, &y2);

        ll m = (y2 - y1) / M;
        tree.insert(0, M, Line(m, y1));
    }

    for(int x = 0; x <= M; x++){
        ll res = tree.query(x);
        printf("%lld%c", (res == -INF ? -1 : res), (" \n")[x==M]);
    }
}

```

Line Segments Trace II

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Line {
    ll a, b;

    Line() : a(0), b(-INF) {}
    Line(ll a, ll b) : a(a), b(b) {}

    ll get(ll x){

```

```

        return a * x + b;
    }
};

struct Node {
    Line line = Line();
    Node* lc = nullptr;
    Node* rc = nullptr;
};

struct LiChaoTree {
    Node* root;
    ll size;
    LiChaoTree(ll size) : root(nullptr), size(size) {}

private:
    void contained_insert(Node* &node, ll tl, ll tr, Line line){
        if(node == nullptr) node = new Node();
        if(node->line.get(tl) < line.get(tl)) swap(node->line, line);
        if(node->line.get(tr) >= line.get(tr)) return;
        if(tl == tr) return;

        ll tm = (tl + tr) / 2;
        if(node->line.get(tm) > line.get(tm)) {
            contained_insert(node->rc, tm+1, tr, line);
        } else {
            swap(node->line, line);
            contained_insert(node->lc, tl, tm, line);
        }
    }

    void insert(Node* &node, ll tl, ll tr, ll l, ll r, Line line){
        if(tr < l || r < tl || tl > tr || l > r) return;
        if(node == nullptr) node = new Node();
        if(l <= tl && tr <= r) return contained_insert(node, tl, tr, line);

        ll tm = (tl + tr) / 2;
        insert(node->lc, tl, tm, l, r, line);
        insert(node->rc, tm+1, tr, l, r, line);
    }

    ll query(Node* &node, ll tl, ll tr, ll x) {
        if(node == nullptr) return -INF;
        if(tl == tr) return node->line.get(x);

        ll res = node->line.get(x);
        ll tm = (tl + tr) / 2;
        if(x <= tm) res = max(res, query(node->lc, tl, tm, x));
        else res = max(res, query(node->rc, tm+1, tr, x));

        return res;
    }

public:
    void insert(ll l, ll r, Line line) {
        insert(root, 0, size-1, l, r, line);
    }

    ll query(ll x) {
        return query(root, 0, size-1, x);
    }
};

int main(){
    int N, M;
    scanf("%d %d", &N, &M);
    LiChaoTree tree(M+1);

    for(int i = 0, x1, y1, x2, y2; i < N; i++){

```

```

scanf("%d %d %d %d", &x1, &y1, &x2, &y2);

ll m = (y2 - y1) / (x2 - x1);
ll b = y1 - m * x1;
tree.insert(x1, x2, Line(m, b));
}

for(int x = 0; x <= M; x++){
    ll res = tree.query(x);
    printf("%lld%c", (res == -INF ? -1 : res), (" \n")[x==M]);
}
}

```

Lines and Queries I

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f;
const ll maxX = 1e5+5;

struct Line {
    ll a, b;

    Line() : a(0), b(-INF) {}
    Line(ll a, ll b) : a(a), b(b) {}

    ll get(ll x){
        return a * x + b;
    }
};

struct Node {
    Line line = Line();
    Node* lc = nullptr;
    Node* rc = nullptr;
};

struct LiChaoTree {
    Node* root;
    ll size;
    LiChaoTree(ll size) : root(nullptr), size(size) {}

private:
    void contained_insert(Node* &node, ll tl, ll tr, Line line){
        if(node == nullptr) node = new Node();
        if(node->line.get(tl) < line.get(tl)) swap(node->line, line);
        if(node->line.get(tr) >= line.get(tr)) return;
        if(tl == tr) return;

        ll tm = (tl + tr) / 2;
        if(node->line.get(tm) > line.get(tm)) {
            contained_insert(node->rc, tm+1, tr, line);
        } else {
            swap(node->line, line);
            contained_insert(node->lc, tl, tm, line);
        }
    }

    void insert(Node* &node, ll tl, ll tr, ll l, ll r, Line line){

```

```

    if(tr < l || r < tl || tl > tr || l > r) return;
    if(node == nullptr) node = new Node();
    if(l <= tl && tr <= r) return contained_insert(node, tl, tr, line);

    ll tm = (tl + tr) / 2;
    insert(node->lc, tl, tm, l, r, line);
    insert(node->rc, tm+1, tr, l, r, line);
}

ll query(Node* &node, ll tl, ll tr, ll x) {
    if(node == nullptr) return -INF;
    if(tl == tr) return node->line.get(x);

    ll res = node->line.get(x);
    ll tm = (tl + tr) / 2;
    if(x <= tm) res = max(res, query(node->lc, tl, tm, x));
    else res = max(res, query(node->rc, tm+1, tr, x));

    return res;
}

public:
    void insert(ll l, ll r, Line line) {
        insert(root, 0, size-1, l, r, line);
    }

    ll query(ll x) {
        return query(root, 0, size-1, x);
    }
};

int main(){
    LiChaoTree tree(maxX+1);

    int T;
    scanf("%d", &T);
    for(int t = 0, type, a, b, x; t < T; t++){
        scanf("%d", &type);
        if(type == 1){
            scanf("%d %d", &a, &b);
            tree.insert(0, maxX, Line(a, b));
        } else if(type == 2){
            scanf("%d", &x);
            ll res = tree.query(x);
            if(res == -INF) printf("NO\n");
            else printf("%lld\n", res);
        }
    }
}

```

Lines and Queries II

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f3f;
const ll maxX = 1e5+5;

struct Line {

```

```

    ll a, b;

    Line() : a(0), b(-INF) {}
    Line(ll a, ll b) : a(a), b(b) {}

    ll get(ll x){
        return a * x + b;
    }
};

struct Node {
    Line line = Line();
    Node* lc = nullptr;
    Node* rc = nullptr;
};

struct LiChaoTree {
    Node* root;
    ll size;
    LiChaoTree(ll size) : root(nullptr), size(size) {}

private:
    void contained_insert(Node* &node, ll tl, ll tr, Line line){
        if(node == nullptr) node = new Node();
        if(node->line.get(tl) < line.get(tl)) swap(node->line, line);
        if(node->line.get(tr) >= line.get(tr)) return;
        if(tl == tr) return;

        ll tm = (tl + tr) / 2;
        if(node->line.get(tm) > line.get(tm)) {
            contained_insert(node->rc, tm+1, tr, line);
        } else {
            swap(node->line, line);
            contained_insert(node->lc, tl, tm, line);
        }
    }

    void insert(Node* &node, ll tl, ll tr, ll l, ll r, Line line){
        if(tr < l || r < tl || tl > tr || l > r) return;
        if(node == nullptr) node = new Node();
        if(l <= tl && tr <= r) return contained_insert(node, tl, tr, line);

        ll tm = (tl + tr) / 2;
        insert(node->lc, tl, tm, l, r, line);
        insert(node->rc, tm+1, tr, l, r, line);
    }

    ll query(Node* &node, ll tl, ll tr, ll x) {
        if(node == nullptr) return -INF;
        if(tl == tr) return node->line.get(x);

        ll res = node->line.get(x);
        ll tm = (tl + tr) / 2;
        if(x <= tm) res = max(res, query(node->lc, tl, tm, x));
        else res = max(res, query(node->rc, tm+1, tr, x));

        return res;
    }
};

public:
    void insert(ll l, ll r, Line line) {
        insert(root, 0, size-1, l, r, line);
    }

    ll query(ll x) {
        return query(root, 0, size-1, x);
    }
};

```

```

int main(){
    LiChaoTree tree(maxX);

    int T;
    scanf("%d", &T);
    for(int t = 0, type, a, b, l, r, x; t < T; t++){
        scanf("%d", &type);
        if(type == 1){
            scanf("%d %d %d %d", &a, &b, &l, &r);
            tree.insert(l, r, Line(a, b));
        } else if(type == 2){
            scanf("%d", &x);
            ll res = tree.query(x);
            if(res == -INF) printf("NO\n");
            else printf("%lld\n", res);
        }
    }
}

```

Area of Rectangles

```

#include

using namespace std;
typedef long long ll;
typedef array<Operation> Operation;
const int maxN = 1e5;
const int SZ = 9e6;

int N, lo[SZ], hi[SZ];
ll area, delta[SZ], score[SZ];
Operation op[2*maxN];

int len(int i){
    return hi[i]-lo[i]+1;
}

void pull(int i){
    if(lo[i] == hi[i]) score[i] = (delta[i] > 0 ? 1 : 0);
    else score[i] = (delta[i] > 0 ? len(i) : score[2*i] + score[2*i+1]);
}

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    if(l == r) return;
    int m = l+(r-l)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
}

void increment(int i, int l, int r, ll val){
    if(l > hi[i] || r < lo[i]) return;
    if(l <= lo[i] && hi[i] <= r){
        delta[i] += val;
        pull(i);
        return;
    }
    increment(2*i, l, r, val);
    increment(2*i+1, l, r, val);
    pull(i);
}

```

```
ll query(){
    return score[1];
}

int main(){
    scanf("%d", &N);
    for(int i = 0, a, b, c, d; i < N; i++){
        scanf("%d %d %d %d", &a, &b, &c, &d);
        op[2*i] = {1, b, a+1, c};
        op[2*i+1] = {-1, d, a+1, c};
    }
    sort(op, op+2*N, [](Operation A, Operation B){
        return (A[1] == B[1] ? A[0] < B[0] : A[1] < B[1]);
    });

    build(1, -1e6-5, 1e6+5);
    int lst = -1e6;
    for(int i = 0; i < 2*N; i++){
        int t = op[i][0], y = op[i][1], x1 = op[i][2], x2 = op[i][3];
        area += (y-lst) * query();
        increment(1, x1, x2, t);
        lst = y;
    }

    printf("%lld\n", area);
}
```
